



Polymarket v2: Oracle Aggregator & Reporters

Security Review

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Security Researcher

July 28, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Unconstrained <code>00ReporterModule</code> can change immutable UMA reporter and break existing requests	4
3.1.2	Unregistered neg-risk subconditions can block resolution and desynchronize rule updates	4
3.1.3	Mid-arbitration arbitrator swaps can leave stale resolution state	5
3.1.4	Migrated NegRisk claims prematurely redeemable on spokes	5
3.2	Gas Optimization	6
3.2.1	Redundant <code>_getRequestConfigForRequestId()</code> invocation can be removed	6
3.2.2	Unreachable price validation branch in <code>00ReporterModule</code> should be removed	6
3.2.3	Redundant disputer threshold guards in oracle configuration setters should be removed	6
3.2.4	Superfluous condition index handling in <code>OracleAggregator</code> finalization	7
3.3	Informational	7
3.3.1	<code>OracleAggregator</code> validation should be combined to make the implicit active invariant explicit	7
3.3.2	Rule updates made before Optimistic Oracle registration causes <code>OracleAggregator</code> and UMA rule histories to diverge	8
3.3.3	Result validation bypasses event-scoped request configuration	9
3.3.4	Codeless module registration can grant EOAs direct oracle authority	9
3.3.5	<code>OracleAggregator.disputeResult()</code> threshold check uses overly defensive inequality	9
3.3.6	Product specification update emits non-canonical name	10

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Jul 13th to Jul 17th the Cantina team conducted a review of [polymarket-v2](#) on commit hash [6997dd82](#). The team identified a total of **14** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	1	3
Gas Optimizations	4	4	0
Informational	6	3	3
Total	14	8	6

2.1 Scope

The security review had the following components in scope for [polymarket-v2](#) on commit hash [6997dd82](#):

```
src
├── libraries
│   └── Ids.sol
├── modules
│   ├── NegRiskModule.sol
│   └── abstract
│       └── ModuleErrors.sol
├── oracle
│   ├── OracleAggregator.sol
│   ├── abstract
│   │   ├── OracleAggregatorErrors.sol
│   │   ├── OracleAggregatorEvents.sol
│   │   └── OracleModuleBase.sol
│   ├── interfaces
│   │   ├── IArbitratorModule.sol
│   │   ├── IBinaryReporter.sol
│   │   ├── IDisputerModule.sol
│   │   ├── IOracleAggregator.sol
│   │   └── IReporterModule.sol
│   ├── libraries
│   │   └── OptimisticOraclePayoutLib.sol
│   ├── mixins
│   │   ├── Auth.sol
│   │   ├── MarketDataRegistry.sol
│   │   └── Pausable.sol
│   └── modules
│       ├── OOReporterModule.sol
│       └── reporters
│           └── EOReporterModule.sol
```

3 Findings

3.1 Low Risk

3.1.1 Unconstrained `00ReporterModule` can change immutable UMA reporter and break existing requests

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: `00ReporterModule` inherits `OracleModuleBase` which restricts upgrades to the contract owner. The UMA `00Reporter` address is immutable in the `00ReporterModule` implementation bytecode, but `_authorizeUpgrade()` does not currently enforce that a new implementation uses the same `00Reporter`. A bad upgrade could therefore move the proxy to a new `00ReporterModule` implementation constructed with a different UMA `00Reporter` address, changing which UMA reporter the proxy reads from while preserving existing proxy storage.

As a result, the proxy can retain `requestInitialized[requestId] == true` for requests that were registered against the old UMA `00Reporter`, but after the upgrade `report()` and `finalize()` will call `isRequestResolved()` / `getRequestResolution()` on the new UMA `00Reporter`. If those request IDs were not registered or settled in the new reporter, normal `00Reporter`-based reporting and finalization can revert with `RequestNotRegistered()`, blocking existing requests.

Recommendation: `00ReporterModule` should further extend the `_authorizeUpgrade()` implementation to constrain upgrades to implementations pointing to the same UMA `00Reporter` address.

Polymarket: Acknowledged. Understood, we would not do this. Upgrades are bound by multisig and timelock and accidental updates that ruin state won't slip in.

Cantina Managed: Acknowledged.

3.1.2 Unregistered neg-risk subconditions can block resolution and desynchronize rule updates

Severity: Low Risk.

Context: `00ReporterModule.sol#L159-L163`, `00ReporterModule.sol#L234`.

Description: `OracleAggregator` configuration is event-scoped, while incremental neg-risk resolution is per subcondition. For incremental neg-risk events, it is understood that condition IDs will be used as request IDs for each condition independently on the `OracleAggregator`. It is assumed that all subconditions will be registered with the `00ReporterModule`; otherwise, if only a subset of subconditions is registered then the following issues could occur:

- If `reporterThreshold` requires an `00ReporterModule` vote for a subcondition that was never registered with UMA, proposal creation for that subcondition can be blocked. If `finalizer` is set to `00ReporterModule`, finalization can also be blocked.
- Rule updates for missing subconditions are recorded by the `OracleAggregator` despite silently skipping forwarding to UMA.

In certain circumstances, it may still be possible to finalize markets so long as it is only `NO` subconditions that remain unregistered; however, in the event that all subconditions must resolve to `NO` then this would block synthetic Other from becoming derivable as `YES` since `conditionsResolved[eventId]` would be unable to reach `conditionCount`.

Recommendation: Ensure that operational guardrails are put in place to guarantee that each real sub-condition of an incremental neg-risk event that needs to be reported on is registered with the `00ReporterModule`.

Polymarket: Acknowledged. Real risk here is forgetting or failing to register the YES resolving subcondition. Fortunately that can be done lazily.

Cantina Managed: Acknowledged.

3.1.3 Mid-arbitration arbitrator swaps can leave stale resolution state

Severity: Low Risk.

Context: `OracleAggregator.sol#L618`.

Description: Mid-arbitration arbitrator swaps can leave stale arbitrator state. `setArbitratorModule()` overwrites `cfg.arbitratorModule` without checking active arbitrations, notifying the new arbitrator, or clearing the old arbitrator. If a request is already `ArbitrationRequested`, the old arbitrator may keep `isActive=true`, while the new arbitrator can still resolve because `resolveResult()` only checks current module identity.

Recommendation: Consider either avoiding arbitrator swaps while any request for the event is in arbitration, or adding an explicit migration path that clears old arbitrator state and initializes the new arbitrator for each affected `requestId`.

Polymarket: Acknowledged. We will make sure the arbitrators can gracefully handle stale states.

Cantina Managed: Acknowledged.

3.1.4 Migrated NegRisk claims prematurely redeemable on spokes

Severity: Low Risk.

Context: `NegRiskModule.sol#L277`.

Description: The `NegRiskModule.getResult` function disables lazy sibling result derivation for migration-backed conditions via the `_isMigrationCondition` check. This ensures migration conditions are resolved individually through the legacy CTF path to settle backing collateral on the hub.

However, because migration events are only registered on the hub, `_isMigrationCondition` always returns `false` on spoke chains. If a migrated event has one condition resolve `YES` on the hub and the result is bridged, the spoke will lazily evaluate unresolved sibling migration conditions as `NO` even though the hub would refuse to resolve them. Users holding these bridged migrated positions can then redeem them on spokes before their corresponding legacy conditions have resolved and settled on the hub.

Recommendation: Block bridging of migration positions on the hub until they have been resolved. This can be implemented by overriding `burnFromBridge` in `NegRiskModule.sol`.

```
function example_canBridgePosition(PositionId positionId) public view returns (bool) {
    if (_isMigrationCondition(positionId.conditionId())) {
        return result[conditionId].length == 2;
    }
    return true;
}
```

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Migration subconditions can no longer be bridged to spokes until every real condition in the migrated event has been explicitly resolved on the hub.

3.2 Gas Optimization

3.2.1 Redundant `_getRequestConfigForRequestId()` invocation can be removed

Severity: Gas Optimization.

Context: [OracleAggregator.sol#L351-L352](#), [OracleAggregator.sol#L901-L905](#).

Description: `_getRequestConfigForRequestId()` is called within `_requireRuleEligible()`, however it is not needed to check the resolution state, plus the only invocation of `_requireRuleEligible()` already calls `_getRequestConfigForRequestId()` immediately beforehand.

Recommendation: Remove the redundant invocation.

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Verified. The redundant invocation has been removed and the `_requireRuleEligible()` documentation has been updated.

3.2.2 Unreachable price validation branch in `OOReporterModule` should be removed

Severity: Gas Optimization.

Context: [OOReporterModule.sol#L283](#).

Description: The outer return present in `OOReporterModule._isValidPrice()` cannot be reached in the current implementation as `OracleAggregator.initializeRequest` requires `resultLength == 1`, and atomic neg-risk is handled before `_isValidPrice()` is executed.

Recommendation: Consider removing the unreachable code and simplifying the preceding conditional logic.

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Verified. The unreachable code has been removed and the conditional logic simplified.

3.2.3 Redundant disputer threshold guards in oracle configuration setters should be removed

Severity: Gas Optimization.

Context: [OracleAggregator.sol#L585](#), [OracleAggregator.sol#L617](#), [OracleAggregator.sol#L645](#).

Description: `OracleAggregator.initializeRequest()` already requires `disputerThreshold >= 1` and a nonzero arbitrator, so the conditional logic present in `setArbitratorModule()`, `addDisputerModules()`, and `setLivenessWindow()` should always execute. The associated dev comments are therefore outdated and should also be updated.

Recommendation: Remove all instances of `if (cfg.disputerThreshold > 0)` since this is already guaranteed.

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Verified. The redundant guards have been removed.

3.2.4 Superfluous condition index handling in `OracleAggregator` finalization

Severity: Gas Optimization.

Context: `OracleAggregator.sol#L686`, `OracleAggregator.sol#L727-L730`, `OracleAggregator.sol#L768-L770`.

Description: `OracleAggregator._finalizeConditions()` caches the condition index in `startIndex`; however, this cache is superfluous and leftover from a previous implementation. A cleaner version would be:

```
uint256[] memory binaryResult = new uint256[](2);
uint256 conditionIndex;

if (_cfg.marketType == MarketType.ATOMIC_NEGRISK) {
    binaryResult[0] = OptimisticOraclePayoutLib.RESULT_DENOMINATOR;
    conditionIndex = _result[0];
} else {
    uint256 value = _result[0];
    binaryResult[0] = value;
    binaryResult[1] = OptimisticOraclePayoutLib.RESULT_DENOMINATOR - value;
    conditionIndex = cid.conditionIndex();
}

ConditionId reportConditionId = eventId.computeConditionId(conditionIndex);
```

In this way, the branch semantics are made explicit: atomic chooses the condition index from the winner result; non-atomic chooses it from the request ID.

In fact, this could be simplified even further by branching on each `MarketType` variant. Since `_getRequestConfigForRequestId()` requires an event-level ID for both `BINARY` and `ATOMIC_NEGRISK` market types, `cid.conditionIndex()` must be 0 and thus does not need to be read by `_finalizeConditions()`. This would allow both `startIndex` and `conditionIndex` to be removed.

Similarly, `_validateResult()` already enforces 0 or `RESULT_DENOMINATOR` for incremental neg-risk since the atomic branch returns early, although this could instead be modified as an else/if. Therefore the `RESULT_DENOMINATOR - value` calculation is unnecessary and could be removed if the above recommendation is applied.

Recommendation: Consider removing superfluous variables and simplifying the conditional logic as described above.

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Verified. The `_finalizeConditions()` logic has been simplified.

3.3 Informational

3.3.1 `OracleAggregator` validation should be combined to make the implicit active invariant explicit

Severity: Informational.

Context: `OracleAggregator.sol#L372-L380`, `OracleAggregator.sol#L414-L418`, `OracleAggregator.sol#L478-L485`, `OracleAggregator.sol#L742-L744`, `OracleAggregator.sol#L758-L766`.

Description: The current implementation of `OracleAggregator` intentionally keeps `resolutionStates` sparse and lazily derives `Active` from the existence of a valid request config. This avoids writing `Active` for every initialized request, and especially avoids materializing every subcondition for incremental neg-risk markets. Using `targetContract != address(0)` as a request existence sentinel is

reasonable; however, validation is currently split across both `_getRequestConfigForRequestId()` and `_isActiveStatus()` :

- `_getRequestConfigForRequestId()` proves the request exists and the request ID is valid.
- `_isActiveStatus(None)` only makes sense after that proof, because `None` means “unmaterialized but implicitly active” for an existing request.

This makes `ResolutionStatus.None` context-dependent and easy to misread, not to mention making future maintenance more difficult. Instead, it might be preferable to combine the validation into a single helper that first proves request existence, and then treats `None` as implicitly active. This would improve readability and reduce future maintenance risk without changing the behavior.

For example:

```
function _requireActiveRequest(bytes32 _requestId)
    internal
    view
    returns (EventId eventId, RequestConfig storage cfg, ResolutionState storage state)
{
    (eventId, cfg) = _getRequestConfigForRequestId(_requestId);

    state = resolutionStates[_requestId];

    require(
        state.status == ResolutionStatus.None || state.status == ResolutionStatus.Active,
        RequestNotActive()
    );
}
```

Then `reportResult()`, `disputeResult()`, and `finalize()` can all call such a helper that makes the invariant explicit (`None` is active only after the request config has been validated) instead of separately doing config lookup plus active status check.

Recommendation: Consider combining validation by implementing the helper described above.

Polymarket: Acknowledged. Updated the NatSpec to be more explicit about the behavior.

Cantina Managed: Acknowledged.

3.3.2 Rule updates made before Optimistic Oracle registration causes `OracleAggregator` and UMA rule histories to diverge

Severity: Informational.

Context: `OOReporterModule.sol#L125-L142`, `OOReporterModule.sol#L134-L140`, `OOReporterModule.sol#L155-L157`, `OOReporterModule.sol#L176-L189`, `OracleAggregator.sol#L309-L312`.

Description: Currently, if `initData` is empty and Optimistic Oracle registration is delayed, rule updates made before `createRequest()` is called are recorded in `OracleAggregator` but forwarding is skipped by `OOReporterModule` because `requestInitialized[requestId]` is false. Later calls to `createRequest()` do not replay missed rules updates, so `OracleAggregator` and UMA rule histories can permanently diverge.

Recommendation: Consider disallowing rule updates to unregistered UMA reporter requests, or perhaps instead replay rule updates made before Optimistic Oracle registration to UMA once `createRequest()` is called.

Polymarket: Acknowledged. This is expected, if we init `OOReporterModule` request later, we can init it with updated rules.

Cantina Managed: Acknowledged.

3.3.3 Result validation bypasses event-scoped request configuration

Severity: Informational.

Context: `OracleAggregator.sol#L258-L263`, `OracleAggregator.sol#L723`.

Description: `OracleAggregator._validateResult()` currently requires the result length to be strictly equal to one; however, this array length is already enforced during request initialization and so the validation should be performed against `requestConfigs[ConditionIdLib.from(_requestId).eventId()]` instead.

Recommendation: Avoid repeating hardcoded validation in favor of using the existing `requestConfigs` state.

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Verified. `_validateResult()` has been modified to take the `cfg.resultLength` as an additional parameter.

3.3.4 Codeless module registration can grant EOAs direct oracle authority

Severity: Informational.

Context: `OracleAggregator.sol#L280-L281`, `OracleAggregator.sol#L459-L461`, `OracleAggregator.sol#L669-L672`.

Description: Assuming that registered reporter/disputer modules must be deployed contracts, a codesize check is needed to prevent registrations with no `initData` passing for EOAs. Currently, empty `initData` skips the high-level interface call, so Solidity's implicit `extcodesize` check never runs. The address is still added to `_reporterModules` / `_disputerModules`, and later authorization simply only requires the `msg.sender` address to be contained within the corresponding set. A registered EOA can therefore call `reportResult()`, `disputeResult()`, and `resolveResult()` directly as a "module" as the low-level `IArbitratorModule.onArbitrationTriggered` and `IArbitratorModule.onArbitrationResolved` callbacks silently succeed.

Recommendation: Consider adding explicit codesize validation for every reporter and disputer module before registration, regardless of whether `initData` is provided. If a no-op or sentinel disputer is needed, consider using a deployed no-op module contract rather than a codeless EOA address.

Polymarket: Acknowledged. This is expected, sometimes EOAs might be needed.

Cantina Managed: Acknowledged.

3.3.5 `OracleAggregator.disputeResult()` threshold check uses overly defensive inequality

Severity: Informational.

Context: `OracleAggregator.sol#L431`.

Description: The current condition in `OracleAggregator.disputeResult()` is defensive but could be relaxed to strict equality since it is not possible for the dispute count to surpass the disputer threshold without actually triggering arbitration.

Recommendation: Consider relaxing the condition to strict equality.

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Verified. The condition has been modified to strict equality.

3.3.6 Product specification update emits non-canonical name

Severity: Informational.

Context: [MarketDataRegistry.sol#L189](#).

Description: `MarketDataRegistry.setProductSpecification()` sets `name` on only the first write and it is never subsequently updated; however, the raw `_name` input is still emitted for subsequent invocations that resolve to the same product ID.

`MarketDataRegistry.productId()` lowercases ASCII A-Z before hashing, so product names such as "MLB 2026" and "mlb 2026" resolve to the same product ID. On the first write, `setProductSpecification()` pins the stored canonical display name in `spec.name`; for example, calling `setProductSpecification("MLB 2026", uri)` stores `spec.name == "MLB 2026"`.

On later updates to the same `id`, `spec.name` is intentionally not changed, but the `ProductSpecificationUpdated` event still emits the raw `_name` input. As a result, a later call such as `setProductSpecification("mlb 2026", newUri)` updates the "MLB 2026" product while emitting `name == "mlb 2026"`, even though the canonical on-chain display name remains "MLB 2026". Event consumers may therefore observe a non-canonical product name that does not match `getProductSpecification(...).name`.

Recommendation: Emit the canonical `spec.name` instead of `_name`.

Polymarket: Fixed in [PR 316](#).

Cantina Managed: Verified. The canonical product specification name is now always emitted.